PYTHON 1

Iannick Gagnon
Baccalauréat en Génie mécanique (ÉTS)
Maîtrise et doctorat en Génie logiciel (ÉTS)
Professeur enseignant en informatique – **ÉTS, 2021**
Lead en machine learning – **Eli Health, 2023**

CONTACT

LinkedIn: https://tinyurl.com/5n6b2xtc

Couriel: iannick.gagnon@etsmtl.ca

Plan

- Syntaxe de base
- Écrire du bon/meilleur code
- S'amuser

Variables

```
>>> x = 10                ← déclaration + affectation
>>> id(x)                 ← afficher l'adresse mémoire de x
140737066136648
>>> type(x)               ← afficher le type de x
<class 'int'>
>>> x = x / 2             ← re-affectation (≠ initialisation)
>>> type(x)
<class 'float'>           ← typage dynamique
```

SOURCE: https://docs.python.org/3/library/functions.html

Types

```
>>> type(1)          ← entiers
<class 'int'>
>>> type(1.0)        ← rationnels
<class 'float'>
>>> type(True)       ← booléens
<class 'bool'>
>>> type('a')        ← chaînes de caractères
<class 'str'>
```

SOURCE: https://docs.python.org/3/library/stdtypes.html

Affichage simple (`print`)

```
print(*objects, sep=' ', end='\n')
```

```
>>> x = 0
>>> y = 1
>>> print(x, y)
0 1
>>> print(x, y, sep=',')
0,1
```

SOURCE: https://docs.python.org/3/library/functions.html#print

Opérateurs (+, −, etc.) et promotions

PEMDAS

```
>>> print(1 + 1.0 - 3, type(1 + 1.0 - 3))
-1.0 <class 'float'>
>>> print(1 / 2 * 4, type(1 / 2 * 4))
2.0 <class 'float'>
>>> print((5 // 2) ** 2, type((5 // 2) ** 2))
4 <class 'int'>
>>> print((5 % 2) - True, type((5 % 2) - True))
0 <class 'int'>
```

XR6 - Racines de $y = ax^2 + bx + c$

a $\leftarrow$ 1

b $\leftarrow$ 2

c $\leftarrow$ -1

$r1 \leftarrow \frac{-b-\sqrt{b^2-4ac}}{2a}$

$r2 \leftarrow \frac{-b+\sqrt{b^2-4ac}}{2a}$

Afficher $r1$ et $r2$

Commentaires simples (#)

```
# Paramètres
a = 1
b = 2
c = -1

# Calcul des racines
r1 = (-b - (b**2 - 4 * a * c)**0.5) / (2 * a)
r2 = (-b + (b**2 - 4 * a * c)**0.5) / (2 * a)

# Affichage
print('r1 = ', r1, 'r2 = ', r2)
```

Encapsulation (def)

- Déclarer fonction → **def foo**(...): ...
- **Décalage constant** du corps

```
def calculer_afficher_racines(a,b,c):

    # Calcul des racines
    r1 = (-b-(b**2-4*a*c)**0.5)/(2*a)
    r2 = (-b+(b**2-4*a*c)**0.5)/(2*a)

    # Affichage
    print('r1 = ', r1, 'r2 = ', r2)
```

En-tête

Corps avec instructions

Conventions d'écriture

- **snake_case** pour les **variables/fonctions/classes**
- **SNAKE_CASE** pour les **constantes symboliques**
- Constantes symboliques > valeurs littérales
- Identifiants **significatifs**
- **Verbe d'action** pour les fonctions
- **Aération** horizontale et verticale
- Ne pas écraser les **mots-clés réservés** (lien)

SOURCE: https://docs.python.org/3/reference/lexical_analysis.html#soft-keywords

```python
def rac(a,b,c):
  x = (-a2-(a2**2-4*a1*a3)**0.5)/(2*a1)
  y = (-a2+(a2**2-4*a2*a3)**0.5)/(2*a1)
  print(x,y)
```

```python
def calculer_afficher_racines(a, b, c):

  # Calcul des racines
  racine_discriminant = (b ** 2 - 4 * a * c) ** 0.5
  racine_1 = (-b - racine_discriminant) / (2 * a)
  racine_2 = (-b + racine_discriminant) / (2 * a)

  # Affichage
  print('r1 = ', racine_1, 'r2 = ', racine_2)
```

Ce n'est que le début ...

https://kobzol.github.io/rust/python/2023/05/20/writing-python-like-its-rust.html

Commentaire d'en-tête (docstring)

```
def calculer_afficher_racines(a, b, c):
  """
  Calcule et affiche les racines d'une
  quadratique.

  Arguments:
    a (int): Coefficient du terme x^2.
    b (int): Coefficient du terme x^1.
    c (int): Coefficient du terme x^0.

  Retourne:
    Rien.
  """
  ...
```

Responsabilité

Paramètres et **types**

Ce qui est retourné (ou pas)

SOURCE: https://google.github.io/styleguide/pyguide.html

Retours et intro aux tuples (`return`)

```
def calculer_quotient_reste(dividende, diviseur):
  """
  Effectue la division entière entre deux entiers.

  Arguments:
    dividende (int): Le dividende.
    diviseur (int): Le diviseur.

  Retourne:
    Un tuple contenant:
      - int: Le quotient de la division entière.
      - int: Le reste de la division entière.
  """
  return dividende // diviseur, dividende % diviseur
```

Valeurs séparées par des virgules

```
>>> resultat = calculer_quotient_reste(7, 3)
>>> print(resultat, type(resultat))
(2, 1) <class 'tuple'>
>>> quotient, reste = calculer_quotient_reste(7, 3)
>>> print(quotient, type(quotient), reste, type(reste))
2 <class 'int'> 1 <class 'int'>
```

XR6 – Min et max

Implémentez une fonction nommée **obtenir_min_max** qui est appelée à l'intérieur d'une fonction nommée **afficher_min_max** dont le comportement est le suivant :

```
>>> afficher_min_max(10, 1)
Le minimum vaut 1
Le maximum vaut 10
```

Aide: https://docs.python.org/fr/3.14/library/functions.html

Les tuples (`tuple`)

- Regroupement **ordonné** et **immuable** de **valeurs quelconques séparées par des virgules**

```
>>> un_tuple = 17, 0, 5, True
>>> print(type(un_tuple))
<class 'tuple'>
```

Types mixtes

```
>>> un_autre_tuple = (17, 0, 5, True)
>>> print(type(un_autre_tuple))
<class 'tuple'>
```

```
>>> un_tuple == un_autre_tuple
True
```

Opérateur de comparaison

- Positions désignées par des **entiers consécutifs commençant à zéro:**

```
>>> print(un_tuple[2])
5
```

Opérateur d'accès par indexation

Les listes (`list`)

- En gros, ce sont des "**tuples modifiables**"

```
>>> une_liste = [1, 2, 3, 4]
>>> une_liste[0] = None      ←——— Absence d'une valeur
>>> print(une_liste)
[None, 2, 3, 4]

>>> un_tuple = (1, 2, 3, 4)
>>> un_tuple[0] = None
TypeError: 'tuple' object does not support item assignment
```

Tuple ou liste?

Une question de ...

- **Clarté sémantique**: Signale une hypothèse importante (c.-à-d. ***cette valeur ne change pas***)
- **Intégrité:** Plus difficile d'altérer le contenu par accident
- **Stabilité**: Plus difficile d'insérer du contenu invalide
- **Sécurité**: Moins facile d'insérer du contenu malicieux
- **Performance**: Moins de mémoire; Optimisations possibles (ex: caching)

Les dictionnaires (`dict`)

- Regroupement **ordonné** de paires **clé:valeur** entre **accolades**

```
>>> point = {'x': 0, 'y': 10}
>>> print('x =', point['x'], 'et y =', point['y'])
x = 0 et y = 10
```

- Peut représenter des concepts plus élaborés

Atelier PM

```
>>> coefficients = {'a': 1, 'b': 2, 'c': -1}
>>> print(', '.join(f'{k} = {v}' for k, v in coefficients.items()))
a = 1, b = 2, c = -1
```

Remarque: Pas idéal, mais le monde est parfois imparfait ...

Saveurs d'arguments

```
def foo(p1, p2, ..., *args, n1=v1, n2=v2, ..., **kwargs):
  ...
```

- Paramètres **positionnels** ...
  - explicites: `p1`, `p2`, `...`
  - supplémentaires: `*args` (tuple)
- Paramètres **nommés** (*keyword arguments* → *kwargs*) ...
  - explicites: `n1=v1`, `n2=v2`, `...`
  - supplémentaires: `**kwargs` (dict)

XR6 – Lecture dirigée

Décrivez la fonction suivante à partir de la documentation:

- https://docs.python.org/fr/3.14/reference/compound_stmts.html#the-for-statement
- https://docs.python.org/fr/3.14/reference/expressions.html#slicings
- https://docs.python.org/fr/3.14/reference/simple_stmts.html#augmented-assignment-statements
- https://docs.python.org/fr/3.14/library/stdtypes.html#str

```
def souder(*objets, sep=' ', end='\n'):
  buffer = ''
  for obj in objets[:-1]:
    buffer += str(obj) + sep
  buffer += str(objets[-1]) + end
  return buffer
```

Opérateurs retournant booléens

- Opérateurs de **comparaison**: <, <=, >, >=, ==
  - Devinette: $\pi^e > e^\pi$?

```
>>> from math import pi, exp
>>> pi ** exp(1) > exp(1) ** pi
False
```

- Opérateurs **logiques**: and, or, not
  - ET logique (and): **Si l'un ET l'autre est vrai, sinon faux.**
  - OU logique (or): **Si l'un OU l'autre est vrai, sinon faux.**
  - NON logique (not): **Inversion booléenne.**

XR6 – Morgane veut savoir …

laquelle de ces deux équations est vraie?

[1] $\overline{A \cup \mathrm{B}} \equiv \overline{A} \cap \overline{B}$

[2] $\overline{A \cup \mathrm{B}} \equiv \overline{A} \cup \overline{B}$

Choix: SI-SINON SI-SINON (`if-elif-else`)

- Exécuter du code **si une condition booléenne est vraie**
  - Implique généralement les **opérateurs de comparaison** et/ou **logiques**

```
>>> valeur = 1
>>> if valeur % 2 == 1:
...   print("Impair")
Impair
```

```
>>> valeur = 10
>>> if valeur % 2 == 1:
...   print("Pair")
... elif valeur % 2 == 0:
...   print("Impair")
Pair
```

```
>>> valeur = 1.5
>>> if valeur % 2 == 1:
...   print("Pair")
... elif valeur % 2 == 0:
...   print("Impair")
... else:
...   print("Autre")
Autre
```

Boucle POUR (for)

```python
liste = [1, 1, 2, 3, 5]
for valeur in liste:
  print(f'valeur = {valeur}')
```

```
valeur = 1
valeur = 1
valeur = 2
valeur = 3
valeur = 5
```

```python
liste = [1, 1, 2, 3, 5]
for i in range(1, len(liste) - 1):
  print(f'liste[{i}] = {liste[i]}')
```

```
liste[1] = 1
liste[2] = 2
liste[3] = 3
```

Remarque 1: **f-string** → substitutions littérales du contenu entre accolades

Remarque 2: **range(a, b)** → a, a+1, ..., b-2, b-1

Remarque 3: **len(collection)** → nombre d'éléments (int)

XR6 – Somme (im)paire

Écrivez une fonction nommée **calculer_somme_paire_impaire** qui reçoit une liste ou un tuple et qui retourne:

1. La somme des valeurs paires;
2. La somme des valeurs impaires.

Boucle TANT QUE (`while`)

```
1 liste = [1, 1, 2, 3, 5]
2
3 i = 0
4 while i < len(liste):
5   print(liste[i])
6   i += 1
```

```
1 1
2 1
3 2
4 3
5 5
```

```
1 liste = [1, 1, 2, 3, 5]
2 while liste:
3   print(liste.pop(0))
```

Pas de panique ...

```
1 1
2 1
3 2
4 3
5 5
```

XR6 – Modulo fait maison

Complétez la fonction suivante **sans utiliser l'opérateur modulo (%) ou la division entière (//) :**

```
def modulo(dividende: int, diviseur: int) -> int:
  """
  Calcule le modulo entre le dividende et le diviseur.
  """
  ...
```

Objets (1/2)

- Le **type** *décrit* la structure et les opérations d'une donnée.
- La **classe** *implémente un type* dans un langage orienté objet comme Python.
  - La structure est principalement implémentée par les **propriétés/attributs.**
  - Les opérations sont implémentées par les **méthodes.**
- Un objet est une **instance d'une classe.**

classe instance / **objet**

```
>>> [1, 2] + [2, 4]
[1, 2, 3, 4]
```

Objets (2/2)

- Pour accéder à un attribut ou une méthode d'un objet, nous utilisons **l'opérateur d'accès** qui correspond à un **point** entre le nom de l'instance et de la méthode :

```
un_objet.nom_methode(...)
```

```
>>> liste = [1, 2]
>>> liste.extend([3, 4])
>>> print(liste)
[1, 2, 3, 4]
```

- **En mots:** Nous avons créé une **instance** de la **classe** list et avons appelé la **méthode** .extend().

REMARQUE: En Python, **tout est un objet.**

XR6 – Moyenne et écart-type

Implémentez la function **afficher_moyenne_ecart_type** qui suit:

```
>>> afficher_moyenne_ecart_type()
Saissez une valeur (rien pour quitter): 1
Saissez une valeur (rien pour quitter): 2
Saissez une valeur (rien pour quitter): 3
Saissez une valeur (rien pour quitter): 4
Saissez une valeur (rien pour quitter): 5
Saissez une valeur (rien pour quitter):
Moyenne    : 3.0
Écart-type : 1.41420135623730951
```